Minimal, Resource-Bounded Runtime Verification for a Safety-Critical Automotive ECU

Damian Navarro
BSc (Hons) Software Development
Institute of Information & Communication Technology (IICT)
Malta College of Arts, Science & Technology (MCAST)
Research Design II (2025/2026)

I. Introduction

This section presents the research design for the project *Minimal, Resource-Bounded Runtime Verification for a Safety-Critical Automotive ECU*. It follows the structure of the Research Design II write-up, covering the theme and rationale, research positioning, background, hypothesis, and the overall aim and purpose of the study.

A. Title and Programme Details

Module: MCAST IICT – Research Design II (2025/2026)
Student: Damian Navarro
Programme: BSc (Hons) Software Development
Institute: Institute of Information and Communication Technology (IICT), MCAST

B. Chosen Research

The working title of the research is:

> Minimal, Resource-Bounded Runtime Verification for a Safety-Critical Automotive ECU.

The design is grounded in recent work on **automotive cybersecurity testing**, **runtime verification for CPS/IoT**, and **hardware-anchored trust and attestation** [1]–[4].

C. Theme and Topic Rationale

Theme: Safety and cybersecurity assurance for cyber-physical and IoT-like automotive systems.

Specific topic: Designing a minimal set of runtime monitors for a safety-critical ECU (e.g. braking), implementable on constrained hardware while supporting current automotive cybersecurity and homologation requirements [1], [5].

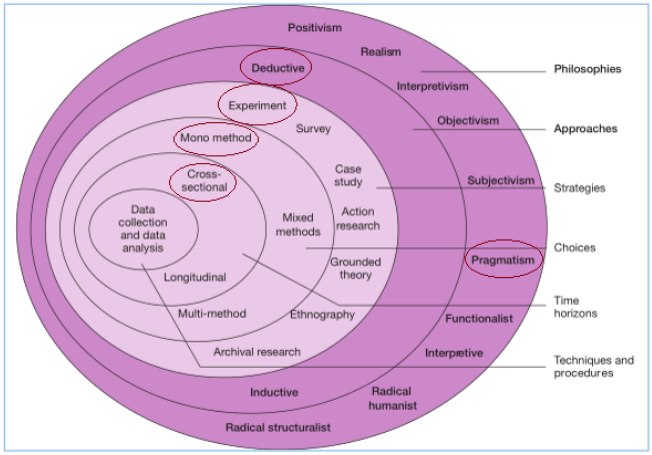
Rationale: New vehicles contain many real-time ECUs connected via CAN-FD buses and a central gateway, and must pass large cybersecurity verification and validation (V&V) test suites under norms such as SAE J3061 and ISO 21434 for homologation [1]. At the same time, IoT-class and automotive controllers are constrained in CPU, memory, power, and area, which limits the applicability of heavyweight security and verification mechanisms [6], [7].

Formal verification and validation surveys for IoT highlight scalability and state-explosion challenges, and recommend lightweight, runtime-oriented approaches for safety- and security-critical systems [5]. Runtime verification (RV) has been shown to be a **pragmatic** technique for CPS and automotive domains, where specifications are represented as automata and monitored during execution [2], [3]. An IoT digital-twin approach demonstrates that only a subset of network traffic needs monitoring to detect communication violations, achieving negligible overhead on constrained devices [3].

To protect keys, firmware, and measurements on resource-constrained platforms, work on hardware trust anchors and remote attestation shows how hardware-based roots of trust and hybrid attestation protocols can provide integrity guarantees while preserving availability [4], [7].

This motivates exploring **how to specify and embed a small, local runtime monitor on a safety-critical ECU** that respects strict resource and timing constraints while contributing to cybersecurity and safety assurance.

D. Positioning and Research Onion



Source: Saunders et al. (2019, p.102)

Fig. 1. The research onion (adapted from Saunders *et al.*) [8].

This study investigates a minimal, resource-bounded runtime verification mechanism for a safety-critical automotive Electronic Control Unit (ECU) using a simulation-based experimental design. The methodology is structured according to the layers of the Saunders *et al.* research onion [8].

a) Research Philosophy: Pragmatism.: The research is grounded in a **pragmatist** philosophy, focusing on developing a solution that is practically effective under realistic resource constraints. The value of the work is judged by whether the proposed runtime verification approach can be integrated into an automotive ECU context and meet performance and safety requirements, rather than by commitment to a single ontological or epistemological stance.

b) Research Approach: Deductive.: A **deductive** approach is adopted. Existing theory and prior work on runtime verification, safety-critical systems, and real-time resource constraints are used to derive expectations about the behaviour of a minimal monitoring mechanism (e.g., that it should incur low overhead while reliably detecting property violations). These theoretically informed expectations are then tested empirically through controlled experiments in a simulated ECU environment.

c) Methodological Choice: Quantitative Mono Method.: The study employs a **quantitative mono-method** design. All primary data are numerical, capturing performance and correctness characteristics of the monitored system. Key variables include execution time overhead, CPU utilisation, memory footprint, and the rate of correctly detected versus missed property violations. These quantitative measures form the basis for assessing whether the proposed mechanism satisfies the targeted resource bounds and safety criteria.

d) Research Strategy: Experiment.: The chosen research strategy is an **experiment**. A runtime verification component is implemented and integrated with a simulated model of a safety-critical automotive ECU. Experimental runs are conducted under controlled conditions, where input workloads, operating scenarios, and injected faults are systematically varied. A baseline configuration without monitoring is used for comparison, enabling the isolation of the effects of the runtime verification mechanism on system performance and fault detection.

e) Time Horizon: Cross-Sectional.: The research adopts a **cross-sectional** time horizon. Data are collected during a finite experimental campaign consisting of multiple simulation runs, each representing a distinct configuration or scenario. The study does not follow the system over an extended real-world operational period; instead, it captures system behaviour at a specific point in time under a range of controlled experimental conditions.

f) Data Collection and Analysis Procedures.: Data collection is performed by instrumenting the simulated ECU environment to record, for each run: cycle-level execution time, CPU load, memory usage attributable to the monitoring component, and whether injected property violations are detected or missed. Faults are injected into the simulated system (for example, by altering parameters or timing) to evaluate the effectiveness of the runtime verification mechanism.

The collected data are analysed using descriptive statistics (such as means and variances of overhead metrics) and comparative analysis between the monitored and unmonitored configurations. This analysis quantifies the performance impact of the monitoring mechanism and its detection capabilities relative to the derived requirements. The results are then interpreted in light of the research objectives to determine whether the minimal, resource-bounded runtime verification approach is suitable for deployment in safety-critical automotive ECU settings.

E. Background to the Research Theme

Modern vehicles are complex cyber-physical systems with many ECUs and a central gateway; cybersecurity V&V must cover thousands of network-level test cases to satisfy norms like ISO 21434, influencing how in-vehicle communication is tested and which attack patterns are considered before road release [1]. Parallel developments in IoT show that pervasive connectivity increases the attack surface, while device constraints hinder deployment of traditional security mechanisms [6].

Formal verification and validation surveys for IoT systems emphasise that exhaustive techniques often do not scale, and advocate combining formal specification with lightweight testing and runtime techniques [5]. In intelligent transport and autonomous vehicles, automata-theoretic, context-aware runtime monitors have been shown to detect unsafe events under varying environmental conditions by composing environment models with system Büchi automata [2].

For IoT communications, a digital-twin-based security architecture uses rule-based runtime verification of parametric events (packets) and demonstrates that monitoring only a subset of traffic can still detect protocol violations with negligible overhead on limited devices [3]. Runtime verification has also been generalised in an algebraic framework that views runs and specification automata through monoidal and semiring structures, enabling incremental, quantitative monitoring and case studies in the automotive domain [9].

On the trust side, surveys of IoT security and remote attestation underline the importance of offloading verification to more capable entities while providing device integrity proofs, especially when local resources are scarce [6], [7]. Hybrid remote attestation protocols specifically target IoT devices, combining randomised memory checks with physical unclonable functions to achieve high availability and low energy usage [4]. Evaluations of hardware trust anchors for automotive applications compare different HTA designs against current and emerging automotive requirements, and identify options that are feasible even for highly constrained controllers [4].

This body of work collectively supports investigating a **minimal, resource-bounded runtime verification design** for a safety-critical ECU that:

- aligns with automotive cybersecurity test practice and regulation [1],
- leverages lightweight RV techniques proven in CPS and IoT contexts [2], [3], [5], and
- is protected and supported by hardware-anchored trust and remote attestation mechanisms [4], [7].

F. Hypothesis

A carefully selected, minimal set of local safety and security properties for a safety-critical automotive ECU can be implemented as runtime monitors with bounded time and memory overhead such that:

- they detect violations relevant to automotive cybersecurity and safety requirements [1]–[3], [5], [10], and
- they remain compatible with the resource and availability constraints of in-vehicle and IoT-class controllers [4], [6], [7].

Formally:

> **H_1:** Minimal, local runtime monitors for a safety-critical ECU can provide meaningful safety and cybersecurity coverage without violating the ECU's real-time and resource constraints.

G. Research Aim and Purpose

Aim: To design and analytically evaluate a minimal, resource-bounded runtime verification monitor for a safety-critical automotive ECU, drawing on automotive cybersecurity V&V practice, CPS/IoT runtime verification techniques, and hardware-anchored trust and attestation mechanisms [1]–[4], [7], [10].

Purpose:

- To derive a method for selecting and formalising a small set of ECU-local properties as automata or algebraic specifications suitable for runtime monitoring under tight resource constraints [2], [3], [5].
- To propose an integration strategy that minimises monitoring overhead through selective observation of events/messages, while still addressing key cybersecurity and safety requirements [1]–[3], [5], [10].
- To outline how hardware trust anchors and remote/hybrid attestation can protect the monitor and its artefacts in automotive-grade, resource-constrained environments [4], [6], [7].

This provides a focused design blueprint that can later be extended and evaluated in broader connected-vehicle and IoT security architectures.

II. Literature Review

Figure 2 summarises how the sources in the research bibliography relate to conceptual themes. Branches group survey, runtime verification, automotive and trust, and methods/tooling; each citation is paired with a short summary (no duplicated sources).

This section reviews how recent studies approach verification, monitoring, and trust for connected, resource-constrained controllers such as automotive ECUs and IoT devices. The discussion is grounded in the sources charted in Fig. 2 [1]–[18].

A. Methodological themes in related work

Several methodological strands recur across these studies. **Survey and taxonomy approaches** synthesise large bodies of technique: Krichen [5] maps formal verification and validation (FV&V) methods for IoT, highlighting where exhaustive analysis gives way to lighter runtime checks, while Williams *et al.* [6] catalogue IoT security challenges under emerging connectivity paradigms. **Empirical and industry-oriented testing** is represented by Ekert *et al.* [1], who relate automotive cybersecurity V&V to structured test campaigns and normative expectations. **Runtime monitoring with formal undertones** appears in Zhang *et al.* [2], who use automata-theoretic environment models for context-aware safety monitoring in intelligent transport settings, and in de Hoz Diego *et al.* [3], who combine an IoT digital twin with rule-based runtime verification over parametric network events, showing that observing only a subset of traffic can still expose protocol violations with low overhead. **Tool-chain and integration studies** include ROSMonitoring 2.0 [7], which extends runtime verification in ROS to services and ordered topics, and work on hardware trust anchors (HTA), where Lorych and Plappert [10] treat authentication for updatable IoT devices and Plappert *et al.* [4] evaluate HTA designs against automotive requirements. Finally, Jakšić *et al.* [9] advance an algebraic view of runtime verification, supporting incremental and quantitative monitoring—a useful reference when specifications are refined iteratively.

B. Academic and non-academic material

Academic sources here include peer-reviewed journals (e.g. [1], [2], [4]–[6]), archival conference proceedings (e.g. [10]), and edited volume chapters (e.g. [3]). Workshop and electronic proceedings such as EPTCS [7] are scholarly but differ from flagship journals in review depth and indexing. **Non-academic or grey** material relevant to automotive work often comprises standards (e.g. ISO/SAE norms referenced in testing practice [1]), vendor security guides, and certification artefacts; these are essential for deployment context but are not substitutes for peer-reviewed evidence. The algebraic framework of Jakšić *et al.* [9] is disseminated via arXiv: it is valuable for technical content but should be read as a preprint unless a later archival version is cited. Distinguishing these layers avoids overstating the evidential weight of grey literature while still acknowledging regulatory drivers.

C. Five recommended peer-reviewed journal articles

For readers prioritising archival journal literature, the following five articles offer strong overlap with automotive cybersecurity, IoT assurance, and runtime-oriented analysis: (1) Ekert *et al.* [1] on automotive cybersecurity V&V testing; (2) Krichen [5] on FV&V for IoT; (3) Williams *et al.* [6] on IoT security and emerging technologies; (4) Plappert *et al.* [4] on hardware trust anchors in automotive settings; (5) Zhang *et al.* [2] on automata-theoretic, context-aware monitoring for autonomous vehicle safety. Together they span testing practice,

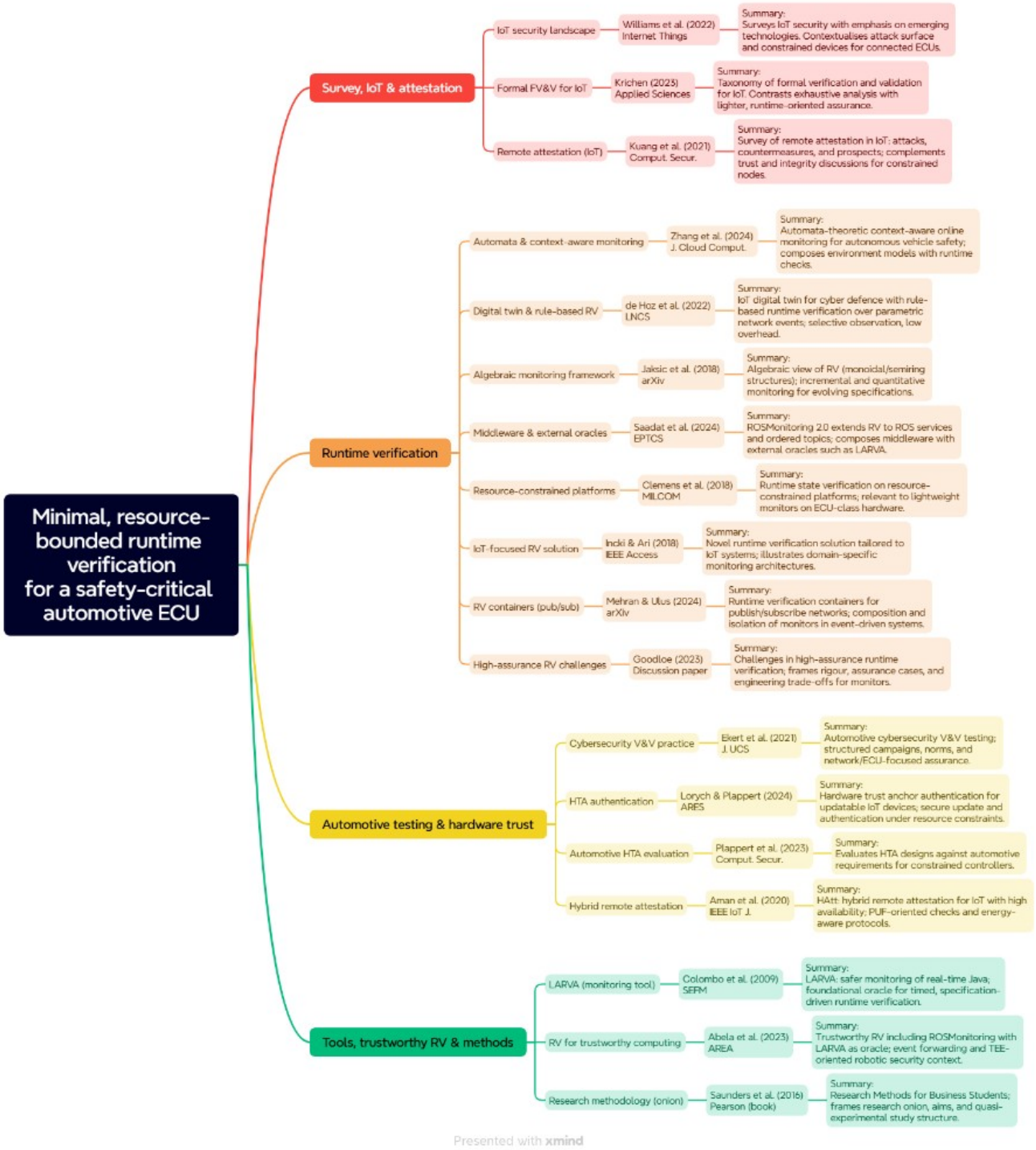


Fig. 2. Literature map covering the full research bibliography (survey/IoT/attestation; runtime verification; automotive testing and hardware trust; tools, trustworthy RV, and methodology). Each citation is paired with a short summary. Sources: Williams *et al.* [6], Krichen [5], Kuang *et al.* [11], Zhang *et al.* [2], de Hoz Diego *et al.* [3], Jakšić *et al.* [9], Saadat *et al.* [7], Clemens *et al.* [12], Incki and Ari [13], Mehran and Ulus [14], Goodloe [15], Ekert *et al.* [1], Lorych and Plappert [10], Plappert *et al.* [4], Aman *et al.* [16], Colombo *et al.* [17], Abela *et al.* [18], Saunders *et al.* [8].

formal-method surveys, security landscape, trust hardware, and concrete monitoring formalisms.

D. Contextualisation for this study

The present research targets a *minimal, resource-bounded* runtime monitor on a safety-related ECU. In that framing,

Ekert *et al.* [1] contextualises the scale and structure of automotive security testing; Williams *et al.* [6] explains why connectivity increases exposure on constrained nodes; and Plappert *et al.* [4] together with Lorych and Plappert [10] ground trust mechanisms that could protect monitors and keys where software-only solutions are insufficient. Zhang *et al.* [2] and de Hoz Diego *et al.* [3] motivate automata- and rule-based runtime checks with selective observation—aligned with limiting bus or signal instrumentation on an ECU. ROSMonitoring 2.0 [7] illustrates how modern tool chains compose middleware events with external oracles, which parallels composing a simulated ECU feed with an external monitor even when the vehicle stack is not ROS-based. Jakšić *et al.* [9] provides vocabulary for quantitative or multi-valued verdicts if requirements later go beyond Boolean pass/fail.

E. Critical comparison and knowledge gaps

The surveyed work **complements** yet **does not fully cover** an integrated ECU-local runtime verification path. Surveys [5], [6] compare techniques at scale but, by design, stop short of prescribing a minimal monitor set for a single braking or powertrain controller. Digital-twin and IoT network studies [3] optimise traffic observation on networked devices, whereas an ECU simulation emphasises *internal* state and sensor plausibility rather than packet subsets. Automata-based context monitoring [2] assumes rich environment models; replicating that fidelity on a student-grade simulation may trade precision for tractability. ROS-oriented tooling [7] demonstrates composable monitoring, but automotive CAN signal semantics and homologation test cases are not identical to ROS topics. HTA work [4], [10] addresses roots of trust and update authentication yet leaves open *what* temporal properties an on-device monitor should enforce under strict CPU/RAM caps. The algebraic framework [9] generalises monitoring semantics but does not resolve deployment questions for legacy ECU binaries.

Knowledge gaps salient for this project include: (i) lack of published, reproducible pipelines that pair *lightweight* ECU signal simulation with declarative monitors such as LARVA for automotive-grade properties; (ii) fragmented guidance on calibrating thresholds (e.g. temperature, RPM) against both safety and cybersecurity abuse cases; (iii) limited reporting on measured overhead when co-locating monitors with attestation or secure boot on class-A/B automotive microcontrollers. Addressing (i)–(iii) experimentally would extend the literature beyond the surveyed sources while building directly on their methodological lessons.

III. Research Methodology

This section describes the planned empirical setup for evaluating a minimal runtime verification layer over a simulated automotive ECU. The approach is **simulation-led** and **monitor-centric**: a script generates time-series ECU signals (as one would derive from CAN frames or internal sensor reads), and an external runtime verifier checks those streams against formalised properties. The design follows a quasi-experimental logic [8]: the simulation acts as a controlled source of behaviours, while monitors provide repeatable verdicts on normality versus violation.

A. Overall approach

The methodology proceeds in four stages: (1) define a small signal model for the ECU under study (coolant temperature, vehicle speed, engine speed, powertrain-related inputs, and electrical health); (2) implement an **ECU simulation script** that emits ordered events or ticks with realistic nominal dynamics, plus injectable fault and attack-like patterns (e.g. bias, spike, impossible co-occurrence); (3) deploy **LARVA** [17] as the monitoring oracle, driven by specifications compiled from automata or temporal rules that encode the properties below; and (4) log monitor verdicts and latency to assess whether violations are detected with acceptable overhead for a resource-bounded target.

Abela *et al.* [18] describe a prototype that couples LARVA with ROSMonitoring for robotic systems:

> As a step towards deploying RV-TEE within robotic systems, we have developed a prototype which combines the RV tool which we have used in our previous works, Larva, with ROSMonitoring to successfully monitor a ROS-based system. Fig. 4 shows how ROSMonitoring listens for relevant events (also known as topics) occurring within the ROS application. These are then forwarded to the Larva monitor, which in turn can send back commands to the system being monitored. As ROSMonitoring is agnostic to the chosen verification system (also referred to as oracle) by design, it was not difficult to combine it with Larva.

—Abela *et al.* [18]

In this study, the ECU simulation script supplies the ordered event stream (analogous to middleware or bus traffic), and LARVA acts as the oracle, without mandating a ROS deployment.

B. Instrumentation: LARVA and flags

LARVA supports specification-driven monitoring of timed behaviour [17]. Each *flag* below is implemented as a monitor property that raises a violation when its predicate becomes false. The five proposed assertion flags are:

1) **Coolant temperature plausibility.** Assert that coolant temperature remains within a calibrated operating band for the current engine load profile and does not jump faster than a maximum physical slew rate. Violations indicate sensor fault, wiring issues, or spoofed frames.
2) **Vehicle speed consistency.** Assert that reported speed changes within bounds per timestep (no instantaneous jumps beyond a configured acceleration limit) and remains non-negative. Detects abnormal dynamics or inconsistent integration of wheel-speed sources.
3) **RPM versus speed coherence.** Assert a gear-aware consistency relation between engine RPM and vehicle

speed (e.g. within tolerances for fixed gear or inferred ratio), flagging implausible combinations that may indicate manipulated signals or model mismatch.

4) **Throttle–brake mutual exclusion.** Assert that demanding significant brake torque is not simultaneously accompanied by high throttle demand above a safety threshold, except where a defined hybrid/regeneration mode explicitly allows it. Captures hazardous driver-assist edge cases or erroneous actuator commands.
5) **Electrical / lubrication sanity (battery voltage or oil pressure).** Assert that supply voltage stays within cranking and running windows for the scenario, and that oil pressure correlates with RPM in a minimal nominal model (e.g. pressure not near zero at sustained high RPM). Flags electrical faults or lubrication anomalies relevant to safe operation.

Each flag is evaluated on the simulated trace; injection campaigns systematically activate single and paired faults to validate that the corresponding LARVA specification fires and that benign traces produce no false alarms under chosen thresholds.

C. Research questions and validation

The following questions guide evaluation: (RQ1) Do the five properties detect the injected abnormal classes with high recall? (RQ2) What is the observed monitoring overhead relative to simulation tick rate on the development hardware? (RQ3) Which properties require threshold tuning to avoid false positives under noisy but legitimate traces? Answers are sought by structured log analysis and repetition across parameterised scenarios, consistent with cybersecurity V&V emphasis on repeatable test cases [1].

D. Limitations

Results apply to the simulation and property set instantiated here; they do not replace homologation testing or formal verification of production ECU binaries. Future work could couple the same monitors with recorded CAN traces or hardware-in-the-loop benches.

References

[1] D. Ekert, J. Dobaj, and A. Salamun, "Cybersecurity verification and validation testing in automotive," *J. Univers. Comput. Sci.*, vol. 27, pp. 850–867, 2021.

[2] Y. Zhang, S. Xu, H.-Y. Chen, U. Bhatti, and M. Huang, "Context-aware environment online monitoring for safety autonomous vehicle systems: an automata-theoretic approach," *J. Cloud Comput.*, vol. 13, 2024.

[3] J. de Hoz Diego, A. Temperekidis, P. Katsaros, and C. Konstantinou, "An iot digital twin for cyber-security defence based on runtime verification," in *Software Engineering and Formal Methods*, ser. LNCS. Springer, 2022, pp. 556–574.

[4] C. Plappert, D. Lorych, M. Eckel, L. Jäger, A. Fuchs, and R. Heddergott, "Evaluating the applicability of hardware trust anchors for automotive applications," *Comput. Secur.*, vol. 135, p. 103514, 2023.

[5] M. Krichen, "A survey on formal verification and validation techniques for internet of things," *Applied Sciences*, 2023.

[6] P. Williams, I. K. Dutta, H. G. Daoud, and M. Bayoumi, "A survey on security in internet of things with a focus on the impact of emerging technologies," *Internet Things*, vol. 19, p. 100564, 2022.

[7] M. G. Saadat, A. Ferrando, L. A. Dennis, and M. Fisher, "Rosmonitoring 2.0: Extending ros runtime verification to services and ordered topics," *EPTCS*, 2024.

[8] M. Saunders, P. Lewis, and A. Thornhill, *Research Methods for Business Students*, 7th ed. Harlow: Pearson, 2016.

[9] S. Jakšić, E. Bartocci, R. Grosu, and D. Ničković, "An algebraic framework for runtime verification," *arXiv*, 2018.

[10] D. Lorych and C. Plappert, "Hardware trust anchor authentication for updatable iot devices," in *Proceedings of the 19th International Conference on Availability, Reliability and Security*, 2024.

[11] B. Kuang, A. Fu, W. Susilo, S. Yu, and Y. Gao, "A survey of remote attestation in internet of things: Attacks, countermeasures, and prospects," *Computers & Security*, 2021.

[12] J. Clemens, R. Pal, and B. Sherrell, "Runtime state verification on resource-constrained platforms," in *Milcom 2018 Track 3 – Cyber Security and Trusted Computing*, 2018.

[13] K. Incki and I. Ari, "A novel runtime verification solution for iot systems," *IEEE Access*, 2018.

[14] A. Mehran and D. Ulus, "Runtime verification containers for publish/subscribe networks," *arXiv*, 2024.

[15] A. Goodloe, "Challenges in high-assurance runtime verification," *none*, 2023.

[16] M. N. Aman, M. H. Basheer, S. Dash, J. W. Wong, J. Xu, H. W. Lim, and B. Sikdar, "Hatt: Hybrid remote attestation for the internet of things with high availability," *IEEE Internet of Things Journal*, 2020.

[17] C. Colombo, G. J. Pace, and G. Schneider, "LARVA: Safer monitoring of real-time java programs (tool paper)," in *Proc. 7th IEEE Int. Conf. Software Engineering and Formal Methods (SEFM)*. IEEE Computer Society, 2009, pp. 33–37.

[18] R. Abela, C. Colombo, A. Curmi, M. Fenech, M. Vella, and A. Ferrando, "Runtime verification for trustworthy computing," *AREA 2023*, 2023.